CONSTRUCTIVE LARGE NUMBERS / BUSYLEAN

Ramified Semantic Worlds

A proposed Contender 7 for BusyLean

Whole-interpreter reflection, self-generated tree indices,
and an explicit 42-fold tower lower bound

Research proposal prepared for Vladimir Reshetnikov
October 12, 2026

Abstract

The incumbent BusyLean construction reflects a finite-type W-language over numeric oracle families and iterates this reflection along the trees the language itself defines. We propose a further step: reflect *entire semantic worlds*. A world supplies a tree-recursion operator accepting an arbitrary seed, an arbitrary countably branching well-founded tree, and a natural argument, together with its own program-generated index. A new language can access the complete operators and indices of a countable family of previously constructed worlds. Its bounded program maximum and its own tree index form a stronger world. We ramify this operation along the preceding world's own index and repeat the resulting world transformation 42 times. The resulting closed natural-number definition is computable in principle. Writing C_6 for the incumbent and $A(x) = (u \mapsto 2^u)^{[x]}(x)$, we prove

$$\boxed{A^{[42]}(C_6) < C_7.}$$

We also prove relative language domination, numerical non-representability, and non-representability of the new tree index. The comparisons do not assume monotonicity of an interpreter in its oracle. Lean definition and proof source, an independent small witness evaluator, and reproducible audit scripts are included. The mathematical argument is complete at the stated level; the Lean source has not been compiled in this authoring environment.

Validation boundary. Nine Python test methods pass, including typed witness checks for all 36 child indices visible to the tower certificate at input 42. No Lean compiler was available. Accordingly, kernel acceptance, axiom-closure checks, and the 15-second/60-second contest budgets are **not certified**. No such success is asserted by this report.

Audited upstream snapshot: `86b360648163f0249eab79539438b6a83dfe8215`.
Exact comparison scope: incumbent Contender 6, proposal P5, and the faithful Contender 5 port. No exact comparison with each separate P1–P4 proposal is claimed.

Contents

1 The result and its scope **3**

2 What the incumbent actually supplies **3**
2.1 Why oracle monotonicity is the wrong comparison principle . . . 4

3 Worlds and a finite semantic signature **4**
3.1 A combinatory extension rather than a second binder mechanism . . . 5
3.2 The enumeration really is finite . . . 6

4 Reflecting whole lower worlds **6**

5 The new language generates its own tree index **7**

6 A seven-level tower certificate **8**
6.1 The witness and its exact type . . . 8

7 World-valued ramification and the final number **9**
7.1 The elementary monotonicity that is actually used . . . 10
7.2 A uniform diagonal family . . . 10

8 Relative language domination and separation **11**

9 Why the number is computable in principle **11**

10 Lean artifacts, audit design, and validation **12**
10.1 Artifact organization . . . 12
10.2 What was actually tested . . . 13
10.3 What remains unverified . . . 13
10.4 The theorem-to-source correspondence . . . 14

11 Research directions and unresolved comparisons **14**
11.1 Complete the kernel and resource audit . . . 14
11.2 Certify P1–P4 at the fixed input . . . 14
11.3 Remove convenience atoms without losing the 42 budget . . . 14
11.4 Replace the inherited seed guard . . . 15
11.5 Analyze world ramification as an operation on function classes . . . 15
11.6 Study observational equivalence and tree transformations . . . 15
11.7 Higher-order world operators and alternative schedules . . . 15
11.8 A reusable constructive reflection framework . . . 15

12 Conclusion **15**

1 The result and its scope

The objective is not to append a successor to the old contestant, nor to assume an uncomputable busy-beaver oracle. It is to describe a stronger constructive mechanism and give a transparent comparison that remains meaningful even when neither number can be evaluated.

Let B denote the repository's existing seed family, called `BusyLean.p5Family`. Let w_0 consist of its existing forest-recursion operator and its existing index. We construct an operation Elev on semantic worlds and set

$$w_{k+1} = \mathrm{Elev}(w_k), \qquad S_k(n) = \mathrm{Score}(w_k, n), \qquad C_7 = S_{42}(42). \tag{1}$$

The base is chosen so that $S_0(n)$ is *definitionally* the incumbent's function `BusyLean.finalFamily n`. In particular, $S_0(42) = C_6$.

The main uniform comparison is stronger than a comparison of two constants: for every $k \in \mathbb{N}$ and every $n \geq 7$,

$$A(S_k(n)) < S_{k+1}(n), \qquad A(x) = (u \mapsto 2^u)^{[x]}(x). \tag{2}$$

Monotonicity of the elementary function A, *not* of any reflection operator, then proves

$$A^{[k]}(S_0(n)) < S_k(n) \qquad (k \geq 1,\ n \geq 7). \tag{3}$$

The bound in the abstract is the instance $k = n = 42$.

There is an additional, more structural reason to regard the proposal as an advance. The new interpreter exposes every lower world's full operator

$$(\text{seed}, \text{tree}, \text{argument}) \longmapsto \text{natural number},$$

not just its value at 42 or its distinguished unary scoring function. Programs can choose a lower world at a computed index, construct a new tree, construct a new seed family, and apply that world's recursion operator to these new arguments. Section 8 proves that the new score eventually exceeds every fixed program in this richer lower-world language. The explicit tower is a convenient certificate, not a complete description of the language's expressive reach.

What is and is not claimed. The repository already supplies exact comparisons $C_5 < C_6$ and $P5 < C_6$ [6]. These combine with our comparison to yield $C_5 < C_7$ and $P5 < C_7$. Its review ranks P1–P4 using informal growth calibrations [7]; that is not an exact proof of their values at the contest input. We do not silently promote that ranking into a new theorem. Similarly, we assign no unproved proof-theoretic ordinal to C_7.

The method uses familiar ingredients—typed interpretation, finite maxima, well-founded recursion, and diagonalization. The proposed contribution is their specific integration into a world-valued ramification construction with explicit, checkable comparison certificates. No claim of global literature novelty is needed for the results below.

2 What the incumbent actually supplies

The audited repository defines a type of well-founded trees

$$\mathcal{T} = \mathrm{WTree}(\mathbb{N}), \qquad \mathsf{leaf} : \mathcal{T}, \qquad \mathsf{node} : (\mathbb{N} \to \mathcal{T}) \to \mathcal{T}.$$

These are inductive, potentially infinitely branching trees, not arbitrary possibly ill-founded graphs. A finite program can denote such a tree by supplying a terminating function for its branches.

The object language has types generated by

$$\tau ::= \mathsf{nat} \mid \tau \to \tau \mid W\tau,$$

with natural-number recursion and W-recursion into every type. The repository interprets raw terms totally: type mismatches receive specified default values. Its finite list `BigNumber.allTerms n` is the syntax enumeration used by the incumbent [2].

Put $\mathcal{F} = \mathbb{N} \to \mathbb{N} \to \mathbb{N}$. The incumbent reflector R_0 satisfies

$$R_0(f, n) = n + 1 + \max\left\{ \max_{i \le n} f(i, n),\ \max_{t \in L_n} \mathrm{evalWith}(f, t) \right\}.$$

The forest recursion derived from it is

$$\begin{aligned} \mathrm{Stage}_0(b, \mathsf{leaf}, n) &= R_0(b, n), \\ \mathrm{Stage}_0(b, \mathsf{node}(a), n) &= R_0(i \mapsto [m \mapsto \mathrm{Stage}_0(b, a(i), m)], n). \end{aligned}$$

The existing index I_0 joins every tree denoted by the old language, with an outer branch for enumeration depth and an inner branch for list position [3, 4].

Consequently, the incumbent is exactly

$$C_6 = \mathrm{Stage}_0(B, I_0, 42).$$

Our base world reuses Stage_0, I_0, and B. The new number's dependency cone is designed not to contain `BusyLean.contender_6`, `BusyLean.finalFamily`, or either proposal-level named contender. This is more than renaming the previous constant: the reused operator accepts arbitrary seeds and arbitrary trees, and the new interpreter makes those parameters available to generated programs.

2.1 Why oracle monotonicity is the wrong comparison principle

A program may inspect an oracle result and branch on it. Even an elementary expression such as "return 100 if the oracle returns zero, otherwise return zero" is not monotone in its oracle. The repository itself explicitly warns against assuming monotonicity of its reflector [5].

We therefore do not argue that replacing each child score by a larger score necessarily makes its parent's reflection larger. Our proofs instead use a single well-typed witness program whose *exact* denotation is known. Its value is one of the finitely many values over which the new reflector maximizes. This approach is unaffected by the behavior of the other programs in the maximum.

3 Worlds and a finite semantic signature

Definition 3.1 (Semantic world)**.** A world is a pair $w = (\mathrm{run}_w, I_w)$, where

$$\mathrm{run}_w : \mathcal{F} \to \mathcal{T} \to \mathbb{N} \to \mathbb{N}, \qquad I_w : \mathcal{T}.$$

Its distinguished score is

$$\mathrm{Score}(w, n) = \mathrm{run}_w(B, I_w, n).$$

The base world is $w_0 = (\mathrm{Stage}_0, I_0)$.

The record deliberately has no field containing its distinguished natural number. It stores the reusable semantic machinery. The type of a runner is itself a finite object-language type, since $\mathcal{F}$, $\mathcal{T}$, and $\mathbb{N}$ all have codes in the existing type grammar.

Fix a family $\boldsymbol{w} : \mathbb{N} \to \mathcal{W}$ of already available worlds, a live numeric forest oracle $f : \mathcal{F}$, and an input n. The extension has the following nine atoms. All types in this table are finite types of the old grammar.

Atom	Type	Denotation
`zero`	$\mathbb{N}$	0
`succ`	$\mathbb{N} \to \mathbb{N}$	Natural successor
`input`	$\mathbb{N}$	Current input n
`oracles`	$\mathcal{F}$	Current numeric forest f
`lowerRun`	$\mathbb{N} \to \mathcal{F} \to \mathcal{T} \to \mathbb{N} \to \mathbb{N}$	$i \mapsto \mathrm{run}_{\boldsymbol{w}(i)}$
`lowerIndex`	$\mathbb{N} \to \mathcal{T}$	$i \mapsto I_{\boldsymbol{w}(i)}$
`seed`	$\mathcal{F}$	Existing seed B
`iterate`	$(\mathbb{N} \to \mathbb{N}) \to \mathbb{N} \to \mathbb{N} \to \mathbb{N}$	Finite iteration
`exp`	$\mathbb{N} \to \mathbb{N}$	$x \mapsto 2^x$

The atoms `iterate` and `exp` are ordinary System T operations. They are included explicitly to keep the comparison certificate short and transparent. They do not smuggle in any large contestant value. Removing these convenience atoms and replacing them by old-language terms would change syntax-depth constants, not the basic construction.

3.1 A combinatory extension rather than a second binder mechanism

The new raw syntax is

$$p ::= \mathsf{old}(t) \mid \mathsf{atom}(a) \mid \mathsf{app}(p, p).$$

An old leaf is interpreted as `BigNumber.interpret [] t`. Thus the entire old closed language is retained unchanged. Application uses a dynamic pair of a type code and a value. If the function has arrow type $\sigma \to \tau$, its argument is checked against σ. Equal type codes transport the argument unchanged; unequal codes supply the specified default value of σ. Applying a non-function yields the natural number zero.

We use exact type equality for this *new* application layer. In particular, a correctly typed tree argument is not reconstructed by the old language's structural tree cast. This makes the seven-level witness below literally denote the old runner on the supplied tree. The semantics inside an old leaf remains the original semantics. This distinction avoids claiming that a new casting convention is definitionally identical to every old cast.

No new lambda syntax is needed. Closed old lambda terms provide typed combinators; application of such combinators to semantic atoms expresses abstraction over the new constants. For example, the standard typed K- and S-combinators are available at all finite types. The implementation therefore avoids any second de Bruijn-level convention or capture-avoidance translation.

3.2 The enumeration really is finite

Let $C_0 = []$, and write L_r for the old finite term list at depth r. With Atoms the fixed nine-element list, define

$$\begin{aligned} C_{r+1} = C_r \mathbin{+\!\!+} \mathsf{atom}(\mathsf{Atoms}) \mathbin{+\!\!+} \mathsf{old}(L_{r+1}) \\ \mathbin{+\!\!+} [\mathsf{app}(p,q) : p \in C_r,\ q \in C_r]. \end{aligned} \tag{4}$$

Duplicates are permitted. Every displayed operation constructs a finite list; there is no search over semantic equality, halting, or arbitrary functions.

Lemma 3.2 (Persistence and application closure)**.** *If $p \in C_r$ and $r \le s$, then $p \in C_s$. Every atom belongs to C_{r+1}. If $p, q \in C_r$, then $\mathsf{app}(p,q) \in C_{r+1}$.*

Proof. Persistence for one step is the first summand of (4); iterate it. The other assertions are the second and fourth summands. □

Arbitrary old terms are *not* unit-cost literals. An old leaf must first occur in L_{r+1}. For completeness, assign type costs $d(\mathsf{nat}) = 1$, $d(\sigma \to \tau) = 1 + \max(d\sigma, d\tau)$, and $d(W\sigma) = 1 + d\sigma$. Give old variables $\mathsf{var}(j)$ cost $j + 2$, zero and successor cost 1, each one-type recursor or tree constructor cost one plus its type cost, application cost one plus the maximum subterm cost, and lambdas and W-recursors cost one plus the maximum of the costs of their annotations and subterms. Induction on types and terms, together with monotonicity of the old enumerations, shows that every old term occurs at its cost and at all larger depths. A further induction on p shows that every finite new program belongs to some C_r. The supplied Lean separation theorems take the explicit membership certificate $p \in C_r$, so they do not need this additional completeness lemma in their dependency cone.

4 Reflecting whole lower worlds

Write $E_{\boldsymbol{w},f,n}(p)$ for the new natural-number evaluator, returning zero when the resulting value has a non-natural type. Define

$$R_{\boldsymbol{w}}(f,n) = n + 1 + \max\left\{ \max_{i \le n} f(i,n),\ \max_{p \in C_n} E_{\boldsymbol{w},f,n}(p) \right\}, \tag{5}$$

with the empty program maximum taken to be zero.

Lemma 4.1 (Reflection inequalities)**.** *For all $\boldsymbol{w}, f, n$, one has $n < R_{\boldsymbol{w}}(f,n)$. If $i \le n$, then $f(i,n) < R_{\boldsymbol{w}}(f,n)$. If $p \in C_n$, then $E_{\boldsymbol{w},f,n}(p) < R_{\boldsymbol{w}}(f,n)$.*

Proof. Every relevant value is bounded by one of the displayed maxima. Adding $n+1 \ge 1$ makes that bound strict. The input bound follows immediately from the nonnegativity of the maximum. □

For an arbitrary reflector $R : \mathcal{F} \to \mathbb{N} \to \mathbb{N}$, define

$$\begin{aligned} \mathrm{Fold}_R(b, \mathsf{leaf}, n) &= R(b,n), \\ \mathrm{Fold}_R(b, \mathsf{node}(a), n) &= R(i \mapsto [m \mapsto \mathrm{Fold}_R(b, a(i), m)], n). \end{aligned} \tag{6}$$

The recursion is structural on the tree. A higher-order program inside R may call the supplied family at dynamically computed indices and arguments, but every such family member refers to a proper subtree of the current tree.

The distinction between two interfaces is essential. The atom `oracles` accesses the current numeric forest in (6). The atoms `lowerRun` and `lowerIndex` access previously constructed semantic worlds. The latter are available with arbitrary newly constructed seeds and trees. They are not confined to the child scores currently used by the numeric forest.

5 The new language generates its own tree index

For a fixed family $\boldsymbol{w}$, interpret a program with the live numeric oracle identically zero and the input zero, and extract a value of type $\mathcal{T}$, using the leaf as a default. Call this tree $T_{\boldsymbol{w}}(p)$. Let $C_k[i]$ be the ith list element, with the zero atom as default. Set

$$I_{\boldsymbol{w}} = \mathsf{node}\,(k \mapsto \mathsf{node}\,(i \mapsto T_{\boldsymbol{w}}(C_k[i])))\,. \tag{7}$$

The new world is

$$\mathrm{Ext}(\boldsymbol{w}) = (\mathrm{Fold}_{R_{\boldsymbol{w}}},\ I_{\boldsymbol{w}}). \tag{8}$$

This is not a circular definition. The evaluator in $T_{\boldsymbol{w}}$ can inspect only the supplied lower worlds and the fixed atoms. It does not contain $\mathrm{Ext}(\boldsymbol{w})$, $R_{\boldsymbol{w}}$, or $I_{\boldsymbol{w}}$ as one of its own semantic atoms. Only after the lower-world evaluation and reflection mechanisms have been defined are the new runner and index packaged together.

Theorem 5.1 (Strict domination of every generated tree)**.** *Fix $\boldsymbol{w}$, a seed b, and $p \in C_k$. There is N such that for all $n \geq N$,*

$$\mathrm{Stage}_0(b, T_{\boldsymbol{w}}(p), n) < \mathrm{Stage}_0(b, I_{\boldsymbol{w}}, n).$$

Proof. Because p is a member of the finite list C_k, choose a list position i with $C_k[i] = p$. Take $N = k + i$. The old forest-reflection inequality says that at an argument n, every child with index at most n has score strictly smaller than its parent. Here $i \leq n$ and $k \leq n$. Apply that inequality first to the inner node in (7), then to the outer node, and compose the two strict inequalities. No comparison between different oracles is used. □

Corollary 5.2 (Non-representability of the index)**.** *For every $p \in C_k$,*

$$T_{\boldsymbol{w}}(p) \neq I_{\boldsymbol{w}}.$$

Proof. Equality would turn Theorem 5.1, at $n = N$, into a strict inequality between a natural number and itself. □

This is a constructive tree diagonalization. It does not need an ordinal notation system or a rank assignment. In an ordinary set-theoretic model one may also explain it by ranks: the new index has a two-edge path to every program-generated tree, so its rank is at least the rank of each such tree plus two. That observation is optional and supplies no numerical fast-growing-hierarchy calibration.

The new index also contains each lower world's distinguished tree. The program

$$\mathsf{app}(\mathsf{lowerIndex}, \bar{i})$$

denotes $I_{\boldsymbol{w}(i)}$ and occurs by depth $i + 2$. Thus index reflection is strictly extending a semantic collection of trees, not merely choosing a larger natural parameter for a fixed notation.

6 A seven-level tower certificate

Let $P(x) = 2^x$, and define finite iteration by

$$f^{[0]}(x) = x, \qquad f^{[k+1]}(x) = f^{[k]}(f(x)).$$

Define

$$A(x) = P^{[x]}(x). \tag{9}$$

For orientation only, $A(0) = 0$, $A(1) = 2$, $A(2) = 16$, and $A(3) = 2^{256}$. For $x \geq 1$, $A(x)$ is at least a tower of x twos with top value 1, because iteration of P is increasing in its starting argument.

6.1 The witness and its exact type

Write $\overline{i}$ for the unary successor program for i, and abbreviate application by juxtaposition. Define

$$\begin{aligned} G_i &= \mathsf{lowerRun}\ \overline{i}\ \mathsf{seed}\ (\mathsf{lowerIndex}\ \overline{i}), \\ B_i &= G_i\ \mathsf{input}, \\ Q_i &= \mathsf{iterate}\ \mathsf{exp}\ B_i\ B_i. \end{aligned}$$

The types are respectively $\mathbb{N} \to \mathbb{N}$, $\mathbb{N}$, and $\mathbb{N}$. At any numeric forest f and input n, exact typing gives

$$G_i(x) = \mathrm{Score}(\boldsymbol{w}(i), x), \qquad E_{\boldsymbol{w},f,n}(Q_i) = A(\mathrm{Score}(\boldsymbol{w}(i), n)). \tag{10}$$

In particular, the value is independent of f.

The occurrence of B_i twice is intentional. Its value supplies both the number of exponential iterations and their starting value. It is not merely a single exponentiation of the lower score.

Lemma 6.1 (Membership certificate)**.** *For each $i \in \mathbb{N}$, $Q_i \in C_{i+7}$.*

Proof. A direct application-depth calculation, using Lemma 3.2, gives the following certificates.

Expression	Certified enumeration depth
$\overline{i}$	$i+1$
$\mathsf{lowerRun}\ \overline{i}$	$i+2$
$\mathsf{lowerRun}\ \overline{i}\ \mathsf{seed}$	$i+3$
$\mathsf{lowerIndex}\ \overline{i}$	$i+2$
G_i	$i+4$
B_i	$i+5$
$\mathsf{iterate}\ \mathsf{exp}$	2
$\mathsf{iterate}\ \mathsf{exp}\ B_i$	$i+6$
Q_i	$i+7$

Whenever two argument depths differ, persistence raises the smaller one to the larger before the application-closure rule is used. No evaluation of C_{i+7} is involved. □

Theorem 6.2 (Whole-world tower bound)**.** *For every $\boldsymbol{w}$, every seed b, every tree a, and all i, n satisfying $i + 7 \leq n$,*

$$A(\mathrm{Score}(\boldsymbol{w}(i), n)) < \mathrm{Fold}_{R_{\boldsymbol{w}}}(b, a, n).$$

In particular,

$$A(\mathrm{Score}(\boldsymbol{w}(i), n)) < \mathrm{Score}(\mathrm{Ext}(\boldsymbol{w}), n).$$

Proof. By the membership certificate and persistence, $Q_i \in C_n$. Reflection strictly exceeds its value. Equation (10) identifies that value for *every* numeric forest. At a leaf the relevant forest is b; at a node it is the family of recursively computed child stages. These two cases are precisely (6). Finally take $b = B$ and $a = I_{\boldsymbol{w}}$. □

The arbitrary-tree and arbitrary-seed quantifiers matter. They allow the next construction to choose a complicated self-generated index without introducing any monotonicity obligation.

7 World-valued ramification and the final number

Definition 7.1 (Anchored ramification)**.** For a base world w, define $H_w : \mathcal{T} \to \mathcal{W}$ by structural recursion:

$$\begin{aligned} H_w(\mathsf{leaf}) &= \mathrm{Ext}(i \mapsto w), \\ H_w(\mathsf{node}(a)) &= \mathrm{Ext}(c), \quad c(0) = w, \quad c(j+1) = H_w(a(j)). \end{aligned}$$

Then set $\mathrm{Elev}(w) = H_w(I_w)$.

At each ramification node, the new interpreter sees the whole semantic worlds produced by all its child subtrees. Branch zero is an explicit anchor retaining the base world. The anchor serves two purposes: it gives a small uniform comparison certificate, and it prevents the proof from needing to locate the base inside an enormous program enumeration. It contains an operator and index, not a named predecessor number.

The following is the central portion of the supplied Lean definition.

```
def ramify (base : World) : Tree → World
  | .leaf => extend (fun _ => base)
  | .node f => extend fun i =>
      match i with
      | 0 => base
      | j + 1 => ramify base (f j)
termination_by structural a => a

def elevate (w : World) : World := ramify w w.index

def levels : Nat → World
  | 0 => baseWorld
  | k + 1 => elevate (levels k)

def contender_7 : Nat := score (levels 42) 42
```

Theorem 7.2 (Uniform ramification margin)**.** *For every world w, every tree a, and every $n \geq 7$,*

$$A(\mathrm{Score}(w, n)) < \mathrm{Score}(H_w(a), n).$$

Proof. Inspect the root constructor of a. If it is a leaf, the lower-world family is constantly w. If it is a node, its anchored family satisfies $c(0) = w$. In either case apply Theorem 6.2 with $i = 0$. The theorem requires no inductive comparison of the recursively generated worlds. □

Substituting $a = I_w$ gives the same inequality for $\mathrm{Elev}(w)$, and hence (2) for every stratum.

7.1 The elementary monotonicity that is actually used

Lemma 7.3. *The function $P(x) = 2^x$ is nondecreasing and satisfies $x \le P(x)$. Consequently A is nondecreasing and $x \le A(x)$.*

Proof. For the inflationary inequality, induction gives $1 \le 2^x$ and

$$x + 1 \le 2^x + 2^x = 2^{x+1}.$$

The same doubling recurrence gives monotonicity of P. Induction on r shows that $P^{[r]}$ is nondecreasing in its starting value. Since $P(x) \ge x$, this iterate is also nondecreasing in r. If $a \le b$, then

$$P^{[a]}(a) \le P^{[a]}(b) \le P^{[b]}(b),$$

which is the required monotonicity of A. Inflationarity follows by repeated application of $P(x) \ge x$, including the zero-iteration case. □

Theorem 7.4 (The explicit 42-fold bound)**.** *For every $n \ge 7$ and $k \ge 1$,*

$$A^{[k]}(S_0(n)) < S_k(n).$$

In particular, $A^{[42]}(C_6) < C_7$.

Proof. Let $b_0 = S_0(n)$ and $b_{r+1} = A(b_r)$. First prove $b_r \le S_r(n)$ by induction. Equality holds for $r = 0$. At the successor step,

$$b_{r+1} = A(b_r) \le A(S_r(n)) < S_{r+1}(n),$$

using Lemma 7.3 and Theorem 7.2. The displayed successor argument is already strict, proving the first assertion. Put $k = n = 42$ and use $S_0(42) = C_6$. □

This is not a fixed-height exponentiation bound. Even the first application of A uses the incumbent's entire value as a tower height, and the theorem iterates this transformation 42 times. The true construction additionally reflects much richer lower-world programs at each level and along its self-generated tree indices.

7.2 A uniform diagonal family

Set $D(n) = S_n(n)$. This is a single total computable function, with $C_7 = D(42)$.

Corollary 7.5 (Domination of every fixed stratum)**.** *For $n \ge 7$ and $k < n$, one has $S_k(n) < D(n)$.*

Proof. Each successor step strictly increases the score because $S_j(n) \le A(S_j(n)) < S_{j+1}(n)$. Compose these inequalities from k to n. □

The diagonalization is over this explicitly constructed hierarchy of worlds, not over all total computable functions or all Lean definitions.

8 Relative language domination and separation

Call a program *forest-oracle-free* when it contains no occurrence of the atom `oracles`. Such a program may still use `lowerRun`, `lowerIndex`, `seed`, `input`, all old-language terms, and arbitrary applications. It can therefore carry out substantial higher-order computations involving lower worlds.

Lemma 8.1 (Independence of the live forest)**.** *If p is forest-oracle-free, then for all f, g,*

$$E_{\boldsymbol{w},f,n}(p) = E_{\boldsymbol{w},g,n}(p).$$

Proof. Induct on the application tree of p. An old leaf ignores both forests. Every permitted atom has the same denotation for f and g. Application preserves equality of both evaluated operands. □

Theorem 8.2 (Domination of the full lower-world language)**.** *If $p \in C_k$ is forest-oracle-free, then for every $n \geq k$,*

$$E_{\boldsymbol{w},0,n}(p) < \mathrm{Score}(\mathrm{Ext}(\boldsymbol{w}), n).$$

Proof. Persistence puts p in C_n. At the root of any evaluation of $\mathrm{Fold}_{R_{\boldsymbol{w}}}$, reflection strictly exceeds the value of p in the current numeric forest. The preceding lemma identifies this value with $E_{\boldsymbol{w},0,n}(p)$ independently of the root forest. Apply this to seed B and index $I_{\boldsymbol{w}}$. □

Corollary 8.3 (No lower-language representation)**.** *No forest-oracle-free $p \in C_k$ satisfies*

$$E_{\boldsymbol{w},0,n}(p) = \mathrm{Score}(\mathrm{Ext}(\boldsymbol{w}), n) \quad \textit{for all } n.$$

Proof. Use $n = k$ in Theorem 8.2. The proposed equality contradicts the strict inequality. □

There is also a direct local version without forest-oracle-freeness. For a fixed external forest f, no $p \in C_k$ can satisfy $E_{\boldsymbol{w},f,n}(p) = R_{\boldsymbol{w}}(f, n)$ for every n, by the same contradiction at $n = k$. The forest-oracle-free theorem is stronger for comparing *world scores*, because a world's internal root forest differs from an externally chosen f.

These theorems are the substantive distinction between the proposal and a simple numerical amplification. The next world does not merely evaluate one chosen formula in lower scores. Its score eventually dominates *every fixed program* of a language that can call the complete lower semantic operators on arbitrary program-generated arguments. Theorems are stated with explicit syntax membership bounds: they do not pretend that a program of unbounded depth is already dominated at input 42.

9 Why the number is computable in principle

A bare statement that a fixed natural number is computable would be nearly vacuous: each fixed natural has a constant program. What is needed here is an effective route from the displayed definition to its value, and preferably a uniform family such as D whose totality is justified.

Theorem 9.1 (Constructive totality)**.** *The definitions in Sections 3–7, starting from the repository's constructive base machinery, give a total effective procedure for $D(n)$ for every $n \in \mathbb{N}$. In particular they define C_7 as a computable natural number.*

Proof. The proof follows the construction's dependency order.

First, type-code equality and the new raw interpreter are structurally recursive procedures on finite codes and finite programs. Old leaves use the existing total W-language interpreter. Each primitive is a total function when its supplied lower worlds are effectively available.

Second, C_n is a finite list produced by natural-number recursion. Every maximum in $R_{\boldsymbol{w}}$ is over that finite list or a finite initial segment of $\mathbb{N}$. It therefore terminates, however large the list and its values may be.

Third, $I_{\boldsymbol{w}}$ is built using two W-node constructors and total branch functions. To request a branch, one computes a finite list lookup and the tree denotation of one terminating program. Constructing the tree does not require materializing all infinitely many branches.

Fourth, $\mathrm{Fold}_{R_{\boldsymbol{w}}}$ recurses structurally on a W-tree. Its recursive functional arguments refer only to proper subtrees. Programs may make enormous numbers of calls and choose their arguments dynamically, but those calls do not introduce a self-reference to the current tree.

Fifth, H_w recurses structurally on its schedule tree and returns a world. For a node, every non-anchor lower world comes from a proper child subtree. The new interpreter sees these lower worlds as already supplied data; it never interprets its own semantic definition. Finally, w_n is obtained by natural-number recursion on n, and $D(n)$ is its score at n.

All operations are therefore combinations of finite recursion, total interpretation, and structural recursion on inductive W-trees. No halting oracle, minimization over an undecidable predicate, or classical selection of a maximizing program occurs. □

Arbitrary parameters versus the closed candidate. An arbitrary set-theoretic element of a function type need not come with an effective presentation. The parametric theorems about $\boldsymbol{w}$ are statements about total semantic functions. The computability assertion concerns *effectively supplied* lower worlds, and ultimately the closed family built from w_0. It does not assert that an arbitrary external mathematical oracle becomes computable merely by appearing in a Lean parameter.

Why there is no diagonal paradox. The lower-world language does not contain its new reflector or its new index as an atom. Corollaries 5.2 and 8.3 show exactly why that boundary is essential. A total evaluator can interpret an explicitly weaker, stratified language without interpreting itself. The finite hierarchy and its diagonal remain within Lean's host theory; they do not supply a total interpreter for every total Lean function.

The intended Lean implementations use ordinary `def` declarations, finite lists, inductive types, and structural recursion. This is the standard termination route described by Lean's language reference [8]. The theoretical argument is distinct from a claim that a particular source file has passed the kernel, which requires actually running the toolchain.

10 Lean artifacts, audit design, and validation

10.1 Artifact organization

The archive contains a standalone Lake project pinned to the audited BusyLean commit and to Lean 4.34.0, the repository's declared version. The principal modules are:

File	Role
`BusyLeanNext/Definitions.lean`	Worlds, exact casts, atoms, evaluator, finite enumeration, reflection, index, ramification, final number, and comparison-witness syntax.
`BusyLeanNext/Proofs.lean`	Enumeration membership, exact witness semantics, tower inequalities, comparisons, language separation, and tree index non-representability.
`test/Audit.lean`	Transitive constant-closure and axiom checks.
`test/Smoke.lean`	Only small primitive equalities and theorem-signature checks; no contender evaluation.

The proofs specialize general inequalities only at the end. They do not ask Lean's simplifier to compute a candidate number or expand a bounded maximum at depth 42. Correctly typed application witnesses are proved at symbolic worlds and inputs, before any enormous world is substituted.

The supplied audit is intended to reject dependency on the predecessor's named constant or final-family wrapper, as well as on `Classical.choice`, `sorryAx`, or any custom axiom. The allowed axioms are exactly `propext` and `Quot.sound`, matching the repository's rule. That is a specification of the audit, not a statement that its output has already been obtained.

10.2 What was actually tested

Nine Python unit-test methods passed. They verify exact simple-fragment types, the depth formula $i + 7$ for 100 indices, absence of the live numeric oracle in the witness, correct routing of lower runners and tree identities, exact small tower values, basic exponential inequalities, exact-cast behavior, wrong-function defaults, and the arithmetic finite-maximum inequality. The semantic witness is exercised for every $i \in \{0, \ldots, 35\}$ at input 42, with four deliberately small lower-world scores. These 144 semantic cases ensure that the same indexed world's runner and tree are used and that the live numeric oracle is not accidentally consulted.

The reference evaluator deliberately does *not* implement or enumerate the complete old W-language. It is an independent check of the small certificate and application layer, not a test of C_7 and not a substitute for formal proof. Its safety limits apply only to this testing program; they are not part of the mathematical definition.

A lexical delimiter check was also performed on the supplied Lean files. This catches only elementary source damage. It proves nothing about typing, termination checking, theorem acceptance, or axiom dependencies.

10.3 What remains unverified

No Lean executable was present in the authoring environment. Attempts to obtain the pinned compiler did not provide an executable toolchain. The included `audit/lean-validation.json` therefore records `status: unavailable` and an empty list of executed Lean phases. There are no fabricated build times or kernel-success logs.

In particular, all of the following are outstanding: acceptance of the source by Lean 4.34.0; successful kernel checking of the comparison proofs; the intended transitive dependency and axiom checks; and compliance with the 15-second definition and 60-second proof budgets. The source contains no placeholder proofs, but it is still *uncompiled formalization source*, not a fully validated contest submission. Elaboration or library-interface repairs may be necessary. Such repairs must preserve the mathematical semantics and be followed by the full audit.

To run the reproducibility checks after installing the pinned toolchain:

```
python3 scripts/check_lean.py
python3 -m unittest discover -s tests -v
```

The Lean script fetches dependencies before timing, removes project and dependency build outputs while retaining Lean core, builds definitions and proofs in separate phases, and then runs the audit and smoke tests. It fails rather than treating a timeout or missing compiler as a pass.

10.4 The theorem-to-source correspondence

Mathematical result	Principal Lean declaration
Witness depth	`towerProgram_mem`
Exact tower denotation	`evalNat_towerProgram`
Any-seed, any-tree margin	`tower_lt_foldReflect`
Anchored ramification margin	`tower_lt_score_ramify`
42-fold incumbent bound	`towerOrbit_contender6_lt_contender7`
Required strict comparison	`contender_6_lt_contender_7`
All lower-language programs	`lower_language_eventually_lt`
No lower-language representation	`no_lower_language_representation`
No self-index representation	`no_programTree_eq_index`
Fixed-stratum diagonal domination	`fixed_level_lt_diagonal`

11 Research directions and unresolved comparisons

11.1 Complete the kernel and resource audit

The immediate engineering objective is to compile this exact source at the pinned version, repair any elaboration mismatches, and publish the complete logs. Definitions and proofs should remain separated so that proof-library imports do not inflate the definition budget. Exact casts and explicit membership certificates should be retained unless their replacements come with semantic-equivalence proofs.

11.2 Certify P1–P4 at the fixed input

A worthwhile next theorem is a source-to-syntax compilation of every earlier proposal into this language, with a verified enumeration-depth certificate. That would replace the repository's informal ranking by exact pointwise comparisons. An asymptotic ordinal estimate is insufficient: eventual domination of functions does not by itself compare their values at 42. This is a concrete missing comparison, not a limitation of the incumbent bound proved here.

11.3 Remove convenience atoms without losing the 42 budget

Translate `iterate` and `exp` into closed old-language terms and prove semantic equality. Then optimize the witness's exact enumeration cost. Similar work could remove the explicit zero and successor atoms, although those are already elementary. The interesting optimization target is not merely fewer source characters; it is a smaller, independently verified syntactic certificate with predictable elaboration cost.

11.4 Replace the inherited seed guard

The base seed inherits the old P5 guard involving the old-language maximum. It is constructive and allowed by the present dependency rule, but it remains a historical comparison device. A fully intrinsic typed embedding of the original System T contender, with an explicit short program certificate, could remove this guard while retaining an exact C_5 comparison.

11.5 Analyze world ramification as an operation on function classes

The language-domination theorem is a precise starting point for a proof-theoretic analysis. One should identify which classes of functionals are represented at each stratum, and establish simulations with explicit size translations before assigning an ordinal calibration. The construction combines tree recursion and interpreter promotion; it should not be assigned a growth ordinal by mechanically multiplying the incumbent's informal estimate.

11.6 Study observational equivalence and tree transformations

Exact type casts retain tree values, whereas the old language's self-casts may reconstruct them extensionally. A logical relation connecting these representations would clarify compiler-correctness results and permit aggressive combinatory optimizations. It would also separate genuine semantic strength from harmless changes in tree presentation or enumeration order.

11.7 Higher-order world operators and alternative schedules

The present number uses 42 repetitions of a world transformation that is already ramified along its own index. One can investigate languages whose atoms expose higher-order operations on worlds, or systematically compare alternative anchored schedules. Such an extension needs a fresh stratification argument: merely inserting the current interpreter into its own atom table would violate the separation theorems and destroy the intended totality boundary.

11.8 A reusable constructive reflection framework

Most arguments above depend only on a finite cumulative syntax enumeration, a total typed evaluator, a W-tree type, and a small set of semantic atoms. Abstracting these hypotheses could turn the seven-level witness method, anchored ramification lemma, and self-index separation theorem into reusable Lean components. Future contestants could then change their object language without re-proving all low-level finite-maximum facts.

12 Conclusion

The proposed number is

$$C_7 = \mathrm{Score}(w_{42}, 42), \qquad w_{k+1} = H_{w_k}(I_{w_k}).$$

Its construction promotes complete lower-level semantic operators into a new total language, generates that language's tree index, and ramifies the result at the level of semantic worlds. The

explicit certificate proves

$$A^{[42]}(C_6) < C_7, \qquad A(x) = (u \mapsto 2^u)^{[x]}(x).$$

The language and tree separation theorems explain why the mechanism is more than one preselected numerical amplification. Computability follows from a stratified dependency graph and structural recursion, not from an oracle or an unsupported normalization assumption. The remaining validation step is concrete and falsifiable: compile the supplied Lean source, execute its dependency audit, and measure it against the repository's resource rules.

References

[1] V. Reshetnikov and contributors. *BusyLean*, audited commit `86b360648163f0249eab79539438b6a83dfe8215`. https://github.com/VladimirReshetnikov/BusyLean. Source retrieved October 12, 2026 (America/Los_Angeles).

[2] BusyLean contributors. *P5 W-language definitions*, `proposals/P5-w-reflection/lean/BigNumber/Definitions.lean`, at the commit in [1].

[3] BusyLean contributors. *Contender 6*, `BusyLean/Contender6.lean`, at the commit in [1].

[4] BusyLean contributors. *Reflection over the W-language*, `BusyLean/Reflection.lean`, at the commit in [1].

[5] BusyLean contributors. *Inequalities for reflection and forest reflection*, `BusyLean/ReflectionFacts.lean`, at the commit in [1].

[6] BusyLean contributors. *Contender 6 comparison theorems*, `BusyLean/Comparison.lean`, at the commit in [1].

[7] BusyLean contributors. *Review of the five incoming proposals*, `proposals/REVIEW.md`, at the commit in [1]. Its ordinal-based rankings are explicitly described as informal.

[8] Lean contributors. *The Lean Language Reference: Recursive Definitions*. https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/. Consulted October 12, 2026. The executable project is pinned to Lean 4.34.0, not to a moving "latest" toolchain.